

SIMPLY FPU

by **Raymond Filiatreault**
Copyright 2003

Chap. 11 Logarithmic and exponential instructions

The FPU instructions covered in this chapter perform logarithmic or exponential operations with the value in the TOP data register ST(0), or with both ST(0) and ST(1).

The logarithmic and exponential instructions covered in this document are (in alphabetical order):

[F2XM1](#) 2 to the X power Minus 1

[FSCALE](#) SCALE ST(0) by ST(1)

[FYL2X](#) $Y * \log_2 X$

[FYL2XP1](#) $Y * \log_2 (X+1)$

FYL2X ($Y * \log_2 X$)

Syntax: **fyl2x** (no operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value,
Underflow, Overflow, Zero divide, Precision

This instruction computes the logarithm base 2 of the value in the TOP data register ST(0) and multiplies it by the value in the ST(1) register. The content of ST(0) must be a non-zero positive number. The content of ST(1) is overwritten with the result and the TOP data register is POPed.

The following tabulation gives the resulting values based on the contents of ST(0) and ST(1). "F" means a finite value between 0 and [INFINITY](#), and "Inv" means an Invalid operation.

ST(0)	$+\infty$	$F > +1$	$+1$	$+1 > F > 0$	± 0	$-F$	$-\infty$
ST(1)							
$+\infty$	$+\infty$	$+\infty$	Inv	$-\infty$	$-\infty$	Inv	Inv
$+F$	$+\infty$	$+F$	$+0$	$-F$	$-\infty$	Inv	Inv
$+0$	Inv	$+0$	$+0$	-0	Inv	Inv	Inv
-0	Inv	-0	-0	$+0$	Inv	Inv	Inv
$-F$	$-\infty$	$-F$	-0	$+F$	$+\infty$	Inv	Inv
$-\infty$	$-\infty$	$-\infty$	Inv	$+\infty$	$+\infty$	Inv	Inv

To explain the Invalid results in the above tabulation,

$\log(+\infty) = +\infty$, therefore $\infty * 0$ is Invalid

$\log(+1) = 0$, therefore $0 * \infty$ is Invalid

$\log(+0) = -\infty$, therefore $-\infty * 0$ is Invalid

$\log(\text{negative number})$ is always Invalid (except -0 which is treated as $+0$)

An Invalid operation exception is detected if either ST(0) or ST(1) is empty, or is a [NAN](#), or the combination of operands is indicated as "Inv" in the above tabulation, setting the related flag in the [Status Word](#). The TOP data register would be POPed and the content of ST(0) (formerly ST(1)) would be overwritten with the [INDEFINITE](#) value.

A **Stack Fault** exception is also detected if either ST(0) or ST(1) is empty, setting the related flag in the Status Word.

A **Denormal** exception is detected when the content of either ST(0) or ST(1) is a [denormalized](#) number or a result is a denormalized number, setting the related flag in the Status Word.

A **Precision** exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **Underflow** or **Overflow** exception will be detected if the result exceeds the range limit of [REAL10](#) numbers, setting the related flag in the Status Word.

A **Zero divide** exception will be detected if the content of ST(0) is $\neq 0$ and the content of ST(1) is a non-zero finite number, setting the related flag in the Status Word.

Because the TOP data register is POPed with this instruction, all other values in data registers would now be in the ST(i-1) register.

This instruction is used for two primary reasons. One of them is for the first step in computing the power of any positive number, i.e. x^y . This must be performed using logarithms according to the following relation:

$$x^y = \text{antilog}[y * \log(x)]$$

Although any logarithm base can be used for the above equation, the obvious base for computing logarithms on the FPU is 2, since it operates strictly with binary numbers. The second step in obtaining the x^y value would then be to compute the antilog base 2 of the resulting $y * \log_2(x)$, the above relationship becoming:

$$x^y = 2^{[y * \log_2(x)]}$$

See the [F2XM1](#) instruction for more details on computing powers of 2.

Typical code to compute the $y * \log_2(x)$ portion if both operands are REAL numbers in memory and no error checking is required would be as follows.

```

fld    exponent_y    ;ST(0)=zzz
                        ;load the exponent first
                        ;->ST(0)=exponent, ST(1)=zzz
fld    number_x      ;then load the x value
                        ;->ST(0)=x value, ST(1)=exponent, ST(2)=zzz
fyl2x                ;ST(0)=y*log2(x), ST(1)=zzz

```

The other primary reason for using the FYL2X instruction is to compute the logarithm of a number with a base other than 2. The two most universal bases for logarithms are:

10 which are usually called common logarithms and represented simply as $\log(x)$ without specifying the base, and

e (the Napierian constant = 2.71828...) usually called natural logarithms and represented as $\ln(x)$.

The relationship between a logarithm base 2 and a logarithm using some other base **b** is as follows:

$$\log_b x = \log_2 x * (\log_2 b)^{-1} = \log_2 x * \log_b 2$$

The FPU has both the $\log(2)$ and $\ln(2)$ as hard-coded constants which can be loaded with the [FLDLG2](#) and [FLDLN2](#) instructions respectively. As an example, the following code could be used to compute the common logarithm of a positive REAL number located in memory.

```

fldlg2                ;ST(0)=log10(2), ST(1)=zzz
fld    real_var        ;ST(0)=real_var, ST(1)=log10(2), ST(2)=zzz

```

```
fyl2x          ;log2(real_var)*log10(2)=>ST(1) and ST(0) is POPed
                ;ST(0)=log10(real_var), ST(1)=zzz
```

FYL2XP1 (Y*Log₂(X+1))

Syntax: **fyl2xp1** (no operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value,
Underflow, Overflow, Precision

This instruction computes the logarithm base 2 of the value in the TOP data register ST(0)+1 and multiplies it by the value in the ST(1) register. The content of ST(1) is overwritten with the result and the TOP data register is POPed. The content of ST(0) must be within the range of $-(1-\frac{1}{2}\sqrt{2})$ to $+(1-\frac{1}{2}\sqrt{2})$, i.e. -0.29289... to +0.29289....The value in ST(1) can range from [-INFINITY](#) to [+INFINITY](#).

Note: The stated range for ST(0) is according to Intel's Software Developer's Manual dated 2003. If the value in ST(0) is outside the acceptable range, an exception may not be detected and the result may be undefined. Although an exception may be detected under some conditions, it should not be relied upon for all conditions. The range limits for ST(0) may have been expanded or even eliminated for newer processors.

An **Invalid operation** exception is detected if either ST(0) or ST(1) is empty or is a [NAN](#), setting the related flag in the [Status Word](#). The TOP data register would be POPed and the content of ST(0) (formerly ST(1)) would be overwritten with the [INDEFINITE](#) value.

A **Stack Fault** exception is also detected if either ST(0) or ST(1) is empty, setting the related flag in the Status Word.

A **Denormal** exception is detected when the content of either ST(0) or ST(1) is a [denormalized](#) number or a result is a denormalized number, setting the related flag in the Status Word.

A **Precision** exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **Underflow** or **Overflow** exception will be detected if the result exceeds the range limit of [REAL10](#) numbers, setting the related flag in the Status Word.

This instruction provides improved accuracy over the [FYL2X](#) instruction when the logarithm of a number very close to 1.0 is required. As an extreme example, if the logarithm of a $1+10^{-20}$ value was needed, that value is beyond the precision of the FPU and it would be rounded to a value of 1.0 resulting in a logarithm of 0. The FYL2XP1 gives a result of $1.44*10^{-20}$, which is very small but finite.

This instruction is useful for the computation of compound interest and annuities. Most equations related to those subjects contain a factor in the form of

$$(1+R)^n$$

where **R** is the interest rate expressed in decimal (which is usually small),
and **n** is the number of periods.

F2XM1 (2 to the X power Minus 1)

Syntax: **f2xm1** (no operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value,
Underflow, Precision

This instruction computes the exponential value of 2 to the power of the value of ST(0) and subtracts 1. The content of ST(0) is overwritten with the result. The content of ST(0) must be within the range of -1.0 to +1.0; if it is outside the acceptable range, the result is undefined but no exception is reported.

An **Invalid operation** exception is detected if ST(0) is empty or is a [NAN](#), setting the related flag in the [Status Word](#). The TOP data would be overwritten with the [INDEFINITE](#) value.

A **Stack Fault** exception is also detected if ST(0) is empty, setting the related flag in the Status Word.

A **Denormal** exception is detected when the content of ST(0) is a [denormalized](#) number or the result is a denormalized number, setting the related flag in the Status Word.

A **Precision** exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **Underflow** exception will be detected if the result exceeds the range limit of [REAL10](#) numbers, setting the related flag in the Status Word.

This instruction is part of the second step in computing the power of any positive number, according to the relationship

$$x^y = 2^{[y * \log_2(x)]}$$

However, because the exponent used for this instruction cannot have an absolute value greater than 1.0, another property of exponents must be used.

$$2^{(a+b)} = 2^a * 2^b$$

where **a** could be the integer portion of the **(a+b)** value, and **b** would be the fractional portion. The [FRNDINT](#) instruction can be used to provide an integer portion which is then used to derive a fractional portion. The [FSCALE](#) instruction is finally used to combine the two powers of 2. Typical code to complete the computation of x^y (started with the [FYL2X](#) instruction) would be as follows.

```

                                ;ST(0)=y*log2(x), ST(1)=zzz
fld st                        ;make a second copy
                                ;ST(0)=y*log2(x), ST(1)=y*log2(x), ST(2)=zzz
frndint                      ;round it to an integer
                                ;ST(0)=int[y*log2(x)], ST(1)=y*log2(x), ST(2)=zzz
fsub st(1),st                ;this will leave only a fractional portion in ST(1)
                                ;ST(0)=int[y*log2(x)], ST(1)=y*log2(x)-int[y*log2(x)], ST(2)=zzz
fxch st(1)                   ;ST(0)=y*log2(x)-int[y*log2(x)], ST(1)=int[y*log2(x)], ST(2)=zzz
f2xm1                        ;get the fractional power of 2 (minus 1)
                                ;ST(0)=2ST(0)-1, ST(1)=int[y*log2(x)], ST(2)=zzz
fld1                          ;ST(0)=1, ST(1)=2ST(0)-1, ST(2)=int[y*log2(x)], ST(3)=zzz
fadd                          ;add the 1 to ST(1) and POP ST(0)
                                ;ST(0)=2ST(0), ST(1)=int[y*log2(x)], ST(2)=zzz
fscale                       ;add the integer in ST(1) to the exponent of ST(0)
                                ;effectively multiplying the content of ST(0) by 2int
                                ;and yielding the final result of xy
                                ;ST(0)=xy, ST(1)=int[y*log2(x)], ST(2)=zzz
fstp st(1)                   ;the content of ST(1) has become useless
                                ;overwrite the content of ST(1) with the result and POP ST(0)
                                ;ST(0)=xy, ST(1)=zzz
```

FSCALE (Scale ST(0) by ST(1))

Syntax: **fscale** (no operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value, Underflow, Overflow, Precision

This instruction adds the truncated integral value of ST(1) to the exponent portion of the value in ST(0). The content of ST(1) remains unchanged. This is effectively a multiplication (or division) of the value in ST(0) by an integral power of 2.

The following tabulation gives the resulting values based on the contents of ST(0) and ST(1). "F" means a finite value between 0 and [INFINITY](#), and "Inv" means an Invalid operation.

ST(0)	$+\infty$	+F	+0	-0	-F	$-\infty$
ST(1)						
$+\infty$	$+\infty$	$+\infty$	Inv	Inv	$-\infty$	$-\infty$
+F	$+\infty$	+F	+0	-0	-F	$-\infty$
+0	$+\infty$	+F	+0	-0	-F	$-\infty$
-0	$+\infty$	+F	+0	-0	-F	$-\infty$
-F	$+\infty$	+F	+0	-0	-F	$-\infty$
$-\infty$	Inv	+0	+0	-0	-0	Inv

To explain the Invalid results in the above tabulation,

$2^{+\infty} = +\infty$, therefore $\infty * 0$ is Invalid

$2^{-\infty} = 0$, therefore $0 * \infty$ is Invalid

An Invalid operation exception is detected if either ST(0) or ST(1) is empty, or is a [NAN](#), or the combination of operands is indicated as "Inv" in the above tabulation, setting the related flag in the [Status Word](#). The TOP data register would be overwritten with the [INDEFINITE](#) value.

A Stack Fault exception is also detected if either ST(0) or ST(1) is empty, setting the related flag in the Status Word.

A Denormal exception is detected when the content of ST(0) is a [denormalized](#) number or the result is a denormalized number, setting the related flag in the Status Word.

A Precision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An Underflow or Overflow exception will be detected if the result exceeds the range limit of [REAL10](#) numbers, setting the related flag in the Status Word.

This instruction provides rapid scaling of values, up or down, by multiples of 2. A positive exponent will scale up while a negative exponent will scale down. For example, to reduce a value by a factor of 32 (i.e. 2^5), a value of -5 would be used. To apply that to an array of REAL4 numbers, the following code is suggested.

```

lea  eax,array_var    ;use EAX as a pointer
mov  ecx,array_size   ;use ECX as a counter
pushd -5              ;store an integer value of -5 on the stack
fild dword ptr[esp]    ;ST(0)=-5

@@:
fld  dword ptr[eax]    ;ST(0)=array value, ST(1)=-5
fscale                    ;ST(0)=(array value)/32, ST(1)=-5
fstp dword ptr[eax]    ;replace with the scaled value and pop ST(0)
                        ;ST(0)=-5
fwait                  ;insure previous instruction is completed
                        ;before modifying EAX
add  eax,4             ;adjust pointer to next REAL4 value
dec  ecx               ;adjust counter
jnz  @B                ;continue scaling all the array values

```

```
;clean-up
    fstp st
    add esp,4
;free the FPU register containing the -5
;readjust the stack pointer for the pushed -5
```

[RETURN TO
SIMPLY FPU
CONTENTS](#)